

Relatório de Continuação da Arquitetura - My Food 2026 P2

Data de Geração: 2026-05-06

Versão: 2.0 (Continuação da Versão 1.0)

Status: Projeto Completo com Sistema de Entregas Implementado

Índice

1. Resumo Executivo
 2. Evolução da Arquitetura
 3. Novas Entidades Adicionadas
 4. Hierarquia Expandida de Usuários
 5. Hierarquia Expandida de Empresas
 6. Novo Manager: Delivery
 7. Nuevos Fluxos de Operação
 8. Nuevas Exceções Adicionadas
 9. Integrations Expandidas
 10. Dados Persistidos - Expansão
 11. Padrões de Design - Adições
 12. Comparação Detalhada: Antes vs Depois
-

Resumo Executivo

A versão 2.0 do projeto **My Food** adiciona um **sistema completo de entregas**, expandindo a funcionalidade de um simples gerenciador de pedidos para uma plataforma de **food delivery** integrada.

Principais Adições:

- **Nova entidade Delivery** para gerenciar entregas de pedidos
 - **Nova classe DeliveryMan** herança de User para entregadores
 - **Sistema de categorização de empresas** (Restaurante, Farmácia, Supermercado)
 - **Novo DeliveryManager** para orquestração de entregas
 - **Fluxos de priorização de pedidos** (Farmácia tem prioridade)
 - **7 novas exceções** para validação de entregas
 - **6 novos métodos na Facade** para operações de delivery
-

Evolução da Arquitetura

Antes (v1.0)

CORE (Coordenação)

 userManager

 enterpriseManager

 productManager

 shoppingCartManager (Nomeado como "ShoppingCartManager")

Depois (v2.0)

CORE (Coordenação)

 userManager

 enterpriseManager

 productManager

 orderManager (Renomeado de ShoppingCartManager)

 deliveryManager [NEW]

Mudanças Principais no Core:

Antes: - Core coordenava 4 managers principais - Nenhuma lógica de delivery

Depois: - Core agora coordena **5 managers** (adicionado DeliveryManager) - DeliveryManager injetado com 3 integrators: UserIntegrator, OrderIntegrator, EnterpriseIntegrator - Método **zerarSistema()** atualizado para resetar também deliveryManager - Método **encerrarSistema()** atualizado para persistir também dados de delivery

Novas Entidades Adicionadas

1. Classe Delivery

Localização: models/delivery/Delivery.java

Responsabilidades: - Representa uma entrega de pedido - Contém dados do cliente, empresa, pedido, entregador e destino - Gerencia lista de produtos entregues

Atributos:

```
private String id;           // UUID gerado pelo sistema
private String client;      // Nome do cliente (de User)
private String enterprise;   // Nome da empresa (de Enterprise)
private String orderId;     // ID do pedido
private String deliveryManId; // ID do entregador (de DeliveryMan)
private String destination;  // Endereço de entrega
private List<String> itensList; // Produtos da entrega
```

Métodos Principais: - `getAttribute(String attribute)`: Retorna atributos específicos - "pedido" → `orderId` - "destino" → `destination` - "produtos" → lista de produtos formatada - Lança `AtributoNaoExiste` para atributos inválidos

Estados Adicionados ao Fluxo: - "entregando" - Pedido em processo de entrega - "entregue" - Pedido já entregue

2. Mudanças em User - DeliveryMan

Localização: `models/users/DeliveryMan.java`

Herança: `User` → `DeliveryMan`

Novos Atributos:

```
private String vehicle;           // moto, carro, bicicleta
private String licensePlate;      // placa do veículo
private List<PairKey<String,String>> enterprises_Name_Address; // Empresas que trabalha
private List<String> deliveryByIdList; // Pedidos normais
private List<String> deliveryByIdPriorityList; // Pedidos de farmácia (prioridade)
```

Novos Métodos:

```
// Adicionar/remover pedidos com prioridade
public void addOrder(String orderId, boolean priority)
public void removeOrder(String orderId, boolean priority)

// Obter primeiro pedido (prioridade primeiro)
public String getFirstOrder() throws NaoExistePedidoParaEntrega

// Gerenciar empresas onde trabalha
public void addEnterprise(PairKey<String,String> nameAddress)
public String getEnterpizesList()

// Getters/Setters para atributos
public String getVehicle()
public String getLicensePlate()
public List<PairKey<String, String>> getEnterprises_Name_Address()
public List<String> getDeliveryByIdList()
public List<String> getDeliveryByIdPriorityList()
```

Construtor Vazio (Importante):

```
public DeliveryMan() {
    enterprises_Name_Address = new ArrayList<>();
    deliveryByIdList = new ArrayList<>();
    deliveryByIdPriorityList = new ArrayList<>();
}
```

Necessário para desserialização XML correta, evitando NullPointerException

Sobrescrita em Facade: - criarUsuario(nome, email, senha, endereco, veiculo, placa) cria DeliveryMan

Hierarquia Expandida de Usuários

Antes (v1.0)

User (Abstrata)
 Client
 Owner

Depois (v2.0)

User (Abstrata)
 Client
 Owner
 DeliveryMan [NEW]

Hierarquia Expandida de Empresas

Antes (v1.0)

Enterprise (Abstrata com genéricos)

Não havia diferenciação por tipo de empresa.

Depois (v2.0)

Enterprise (Abstrata)
 Restaurant
 Atributo: kitchenType (brasileira, italiana, etc)
 Pharmacy [NEW]
 Atributo: aberto24Horas
 SuperMarket [NEW]
 Atributos: abre, fecha, tipoMercado

Mudança Significativa: - **Antes:** Enterprise era genérica, sem diferenciação de tipo - **Depois:** Cada tipo de empresa tem suas próprias responsabilidades e atributos

Exemplos: - Restaurant: tipo + ownerId + name + address + kitchenType - SuperMarket: tipo + ownerId + name + address + abre/fecha/tipoMercado - Pharmacy: tipo + ownerId + name + address + aberto24Horas/numeroFuncionarios

Novo Manager: Delivery

DeliveryManeger

Localização: models/manageres/DeliveryManeger.java

Responsabilidades: - Orquestrar criação de entregas - Validar dados de entrega - Persistir entregas - Integrar com UserManager, OrderManager e EnterpriseManager

Dependências Injetadas:

```
private DeliveryDataManeger deliveryDataManeger;  
private DeliveryValidator deliveryValidator;  
private OrderIntegrator orderIntegrator;  
private UserIntegrator userIntegrator;
```

Métodos Principais:

```
createDelivery(orderId, deliveryManId, destination)
```

```
public String createDelivery(String orderId, String deliveryManId, String destination)  
    throws PedidoNaoEstaPronto, PedidoNaoEncontrado, NaoEUmEntregadorValido
```

- **Fluxo:**

1. Valida requisição (pedido pronto, entregador válido)
2. Recupera informações do pedido via OrderIntegrator
3. Gera UUID para identificador da entrega
4. Cria objeto Delivery com dados coletados
5. Persiste via DeliveryDataManeger
6. Retorna ID da entrega

- **Dados Coletados do Pedido:**

- Cliente (nome)
- Empresa (nome)
- ID do Pedido
- Lista de produtos

```
saveData() / resetData()
```

```
public void saveData() throws SaveError  
public void resetData()
```

Nuevos Fluxos de Operação

Fluxo 1: Cadastrar Entregador em Empresa

```
Facade.cadastrarEntregador(empresa, entregador)
```

```

↓
Core.addDeliveryMan(empresa, entregador)
↓
EnterpriseManager.addDeliveryMan(empresa, entregador)
↓
UserIntegrator.getUserById(entregador) → Verifica se existe
↓
Enterprise.addDeliveryMan(email)
↓
DeliveryMan.addEnterprise(PairKey<name, address>)
↓
EnterpriseDataManeger.saveObject(enterprise)

```

Validações: - Empresa cadastrada - Usuário existe - Usuário é entregador (DeliveryMan) - Não duplicar entregador na mesma empresa

Exceções: - EmpresanaoCadastrada - UsuarioNaoEEntregador - UsuarioNaoExisteException

Fluxo 2: Liberar Pedido para Entrega

```

Facade.liberarPedido(numero)
↓
Core.releaseOrder(numero)
↓
OrderManager.releaseOrder(numero)
↓
Order.setState("pronto")
↓
UserManager.addDeliveryManListOrder(entregadores, pedido, isPriority)
↓
Para cada entregador cadastrado na empresa:
    Recupera DeliveryMan via UserIntegrator
    Adiciona pedido à lista apropriada
        Se farmácia → addOrder(orderId, true) → priorityList
        Se outro → addOrder(orderId, false) → normalList
    Persiste DeliveryMan

```

Lógica de Prioridade: - Pedidos de **Farmácia** são adicionados a `deliveryByIdPriorityList` - Pedidos de **Restaurante/Supermercado** são adicionados a `deliveryByIdList` - Os entregadores verificam prioridade antes quando obtêm pedido

Exceções: - PedidoNaoEncontrado - EmpresanaoCadastrada - LiberarPedidoAberto (pedido já estava liberado) - UsuarioNaoEEntregador - UsuarioNaoExisteException - PedidoJaLiberado

Fluxo 3: Obter Pedido para Entregador

```
Facade.obterPedido(entregador)
↓
Core.getOrderByDeliveryMan(entregador)
↓
UserManager.getOrderByDeliveryMan(entregador)
↓
UserIntegrator.getUserById(entregador) → DeliveryMan
↓
DeliveryMan.getFirstOrder()
    Se priorityList não vazio → retorna primeiro
    Senão se normalList não vazio → retorna primeiro
    Senão lança NaoExistePedidoParaEntrega
```

Priorização: 1. Pedidos de **farmácia** PRIMEIRO (sempre) 2. Depois pedidos normais 3. FIFO (First In, First Out) dentro de cada categoria

Exceções: - `UsuarioNaoEEntregador` - `UsuarioNaoExisteException` - `EntregadorNaoTemEmpresa` (nenhuma empresa cadastrada) - `NaoExistePedidoParaEntrega` (nenhum pedido disponível)

Fluxo 4: Criar Entrega

```
Facade.criarEntrega(pedido, entregador, destino)
↓
Core.createDelivery(pedido, entregador, destino)
↓
DeliveryManager.createDelivery(pedido, entregador, destino)
↓
DeliveryValidator.validateDeliveryRequest(pedido, entregador)
    Verifica se pedido está em estado "pronto"
    Verifica se entregador é válido
    Verifica se entregador não está em outra entrega
↓
OrderIntegrator.getOrderInfo(pedido) → Recupera dados
    Cliente
    Empresa
    Lista de produtos
    ID do pedido
↓
Cria objeto Delivery (id, client, enterprise, orderId, deliveryManId, destination, products)
↓
DeliveryDataManager.saveObject(delivery)
↓
Order.setState("entregando")
```

↓
Retorna deliveryId

Validações: - Pedido está em estado “pronto” - Pedido foi encontrado - Entregador é válido - Entregador não está em outra entrega - Pedido não foi liberado recentemente

Exceções: - PedidoNaoEstaPronto - PedidoNaoEncontrado - NaoEUmEntregadorValido - EntregadorAindaEmEntrega - NaoExistePedidoParaEntrega

Fluxo 5: Obter Atributo de Entrega

```
Facade.getEntrega(id, atributo)
↓
Core.getDeliveryAttributeById(id, atributo)
↓
DeliveryManager.getDeliveryAttributeById(id, atributo)
↓
DeliveryDataManeger.getObjectById(id) → Delivery
↓
Delivery.getAttribute(atributo)
    "cliente" → client name
    "empresa" → enterprise name
    "pedido" → orderId
    "entregador" → deliveryManId (recovers name from UserManager)
    "destino" → destination (ou padrão = cliente address)
    "produtos" → lista formatada
    Inválido → throw AtributoNaoExiste
```

Atributos Disponíveis: - "cliente" → Nome do cliente - "empresa" → Nome da empresa - "pedido" → ID do pedido - "entregador" → Nome do entregador - "destino" → Endereço de entrega - "produtos" → Lista de produtos

Exceções: - AtributoInvalido (vazio ou nulo) - AtributoNaoExiste (atributo inválido) - NaoExisteEntregaId (entrega não encontrada)

Fluxo 6: Obter ID de Entrega por Pedido

```
Facade.getIdEntrega(pedido)
↓
Core.getDeliveryIdbyOrderId(pedido)
↓
DeliveryManager.getDeliveryIdbyOrderId(pedido)
↓
```


`DeliveryDataManeger.findByOrderId(pedido)`
Procura por `orderId` na iteração de deliveries
Retorna `deliveryId` se encontrado
Lança `NaoExisteEntregaId` se não encontrado

Exceções: - `NaoExisteEntregaId` (nenhuma entrega com esse pedido)

Fluxo 7: Finalizar Entrega

`Facade.entregar(entrega)`
↓
`Core.finishDelivery(entrega)`
↓
`DeliveryManager.finishDelivery(entrega)`
↓
`DeliveryDataManeger.getObjectById(entrega) → Delivery`
↓
`Order.setState("entregue")`
↓
Recupera `DeliveryMan` via `UserIntegrator`
Recupera `orderId` da entrega
Remove ordem da lista do entregador
`deliveryDataManeger.removeDeliveryToDeliveryMan(deliveryManId)`
(Libera entregador para novo pedido)
↓
`DeliveryDataManeger.saveObject(delivery + order)`

Estados Transicionados: - "entregando" → "entregue"

Efeito Colateral: - Entregador é removido da lista de ocupados - Entregador pode pegar um novo pedido

Exceções: - `NadaParaSerEntregue` (entrega não existe)

Nuevas Exceções Adicionadas

Exceções de Entrega (7 novas)

1. `NaoExisteEntregaId`
 - Lançada quando: Entrega com ID não existe
 - Usado em: `getEntrega()`, `getIdEntrega()`
2. `NadaParaSerEntregue`
 - Lançada quando: Tentativa de finalizar entrega inexistente
 - Usado em: `entregar()`
3. `NaoEUmEntregadorValido`

- Lançada quando: ID passado não é um entregador válido
 - Usado em: `criarEntrega()`
4. **EntregadorAindaEmEntrega**
 - Lançada quando: Entregador tenta pegar novo pedido enquanto está em entrega
 - Usado em: `criarEntrega()`
 5. **EntregadorNaoTemEmpresa**
 - Lançada quando: Entregador não tem empresa cadastrado
 - Usado em: `obterPedido()`
 6. **NaoExistePedidoParaEntrega**
 - Lançada quando: Nenhum pedido disponível para entregador
 - Usado em: `obterPedido()`, `DeliveryMan.getFirstOrder()`
 7. **PedidoNaoEstaPronto**
 - Lançada quando: Tentativa de criar entrega com pedido não-pronto
 - Usado em: `criarEntrega()`

Mudanças em Exceções Existentes

Nenhuma exceção anterior foi removida. Todas as 41 exceções da v1.0 continuam presentes e são usadas conforme necessário.

Integrations Expandidas

Antes (v1.0)

- `UserIntegrator` - Expõe `userManager`
- `EnterpriseIntegrator` - Expõe `enterpriseManager`
- `ProductIntegrator` - Expõe `productManager`

Depois (v2.0)

```
UserIntegrator
  userIsOwner(id)
  getUserNameById(id)
  getUserById(id) ← USADO PARA RECUPERAR DeliveryMan
  getIdByEmail(email)
```

```
EnterpriseIntegrator
  getEnterpriseById(id)
  Métodos existentes...
  Métodos de gestão de entregadores ← EXPANDED
```

```
ProductIntegrator
  Métodos existentes...
```

```

OrderIntegrator
    getOrderInfo(id) ← CRÍTICO para Delivery
    setOrderState(id, state)
    Métodos existentes...

```

Nova Dependência em DeliveryManager:

```

private UserIntegrator userIntegrator;
private OrderIntegrator orderIntegrator;
private EnterpriseIntegrator enterpriseIntegrator;

```

Dados Persistidos - Expansão

Antes (v1.0)

```

data/
    users_by_id.xml
    users_by_email.xml
    enterprise_*.xml (múltiplos)
    product_*.xml (múltiplos)
    shopping_cart_*.xml (múltiplos)

```

Depois (v2.0)

```

data/
    users_by_id.xml ← Agora contém User, Client, Owner, DeliveryMan
    users_by_email.xml
    enterprise_*.xml (múltiplos)
    product_*.xml (múltiplos)
    order_*.xml (múltiplos)
    delivery_*.xml [NEW]
        delivery_by_id.xml
        delivery_by_order_id.xml

```

Novo DataManager: DeliveryDataManager

Localização: models/database/DeliveryDataManager.java

Estrutura de Dados:

```

private Map<String, Delivery> deliveryById;           // id → Delivery
private Map<String, String> deliveryByOrderId;       // orderId → deliveryId
private String deliveryByIdPath = "data/delivery_by_id.xml";
private String deliveryByOrderIdPath = "data/delivery_by_order_id.xml";

```

Métodos:

```

public void saveObject(Delivery delivery)
public Delivery getObjectById(String id)
public Delivery getByOrderId(String orderId)
public void saveData() throws SaveError
public void resetData()
public void removeDeliveryToDeliveryMan(String deliveryManId)

```

Serialização: - Usa XMLEncoder/XMLDecoder (como os outros DataManager) - Persiste em dois mapas para busca rápida (por ID e por OrderID)

Padrões de Design - Adições

Design Patterns Já Existentes (Mantidos)

1. **Facade Pattern** - Facade.java continua
2. **Strategy Pattern** - Validators continuam
3. **Template Method** - DataManager continua
4. **Factory Pattern** - Core continua (implícito)
5. **Dependency Injection** - Todos os managers
6. **MVC Pattern** - Modelos, Testes, Controladores
7. **Repository Pattern** - DataManagers

Novos Padrões Aplicados

1. State Pattern (Aprimorado)

- **Antes:** “aberto” → “preparando” → “pronto”
- **Depois:** “aberto” → “preparando” → “pronto” → “entregando” → “entregue”

Estados agora gerenciados por: - `Order.setState(state)` - Delivery referencia estado atual do pedido

2. Observer Pattern (Implícito) Quando pedido é liberado:

```
Order.setState("pronto")
```

↓

Todos os DeliveryMen associados recebem o pedido

↓

Adicionado automaticamente a suas listas de entrega

3. Priority Queue Pattern Implementado em DeliveryMan via duas listas:

```

deliveryByIdPriorityList // Farmácia (altas prioridades)
deliveryByIdList         // Outros (baixas prioridades)

```

Simulado com `getFirstOrder()`:

```

if (!deliveryByIdPriorityList.isEmpty()) {
    return deliveryByIdPriorityList.getFirst(); // Prioritário SEMPRE primeiro
}

```

Comparação Detalhada: Antes vs Depois

1. Hierarquia de Usuários

Aspecto	v1.0	v2.0
Classes de Usuário	3 (User, Client, Owner)	4 (+ DeliveryMan)
Atributos de Usuário	Nome, Email, Senha, Endereço	+ Veículo, Placa (DeliveryMan)
Gestão de Empresas	Owners em Owner	DeliveryMan em múltiplas empresas
Lista de Pedidos	Clients têm Order	DeliveryMan tem até 2 listas de pedidos
Priorização	Não	Sim (farmácia vs. outros)

2. Hierarquia de Empresas

Aspecto	v1.0	v2.0
Tipos de Empresa	1 genérico (Enterprise)	3 específicos (Restaurant, Pharmacy, SuperMarket)
Diferenciação	Nenhuma	Por tipo de negócio
Atributos Únicos	—	Restaurant: kitchenType Pharmacy: aberto24Horas, numeroFuncionarios SuperMarket: abre, fecha, tipoMercado

Aspecto	v1.0	v2.0
Lista de Entregadores	Não	Sim (para cada empresa)
Roteamento de Pedidos	Não	Sim (para entregadores específicos)

3. Gestão de Pedidos

Aspecto	v1.0	v2.0
Estados do Pedido	3 (aberto, preparando, pronto)	5 (+ entregando, entregue)
Entrega de Pedido	Finaliza em “pronto”	Continua em “entregando”
Rastreamento	Nenhum	Sim (via Delivery)
Entregador Associado	Não	Sim (DeliveryMan)
Destino de Entrega	Endereço padrão do cliente	Customizável ou padrão
Priorização	Não	Sim (farmácia first)

4. Managers

Manager	v1.0	v2.0
UserManager	Mantido	Expandido
EnterpriseManager	Mantido	Expandido
ProductManager	Mantido	Mantido
OrderManeger	Presente	Renomeado (ShoppingCartManeger → OrderManeger)
DeliveryManeger	Não existe	Novo
Total	4	5

5. Integradores (Integrators)

Integrador	v1.0	v2.0
UserIntegrator	SIM	SIM (expandido)
EnterpriseIntegrator	SIM	SIM (expandido)
ProductIntegrator	SIM	SIM
OrderIntegrator	SIM	SIM (novo uso em DeliveryManager)
Total	3	4 (mesmo número, mas expanded usage)

6. DataManagers (Persistência)

DataManager	v1.0	v2.0
UserDataManage	SIM	SIM (agora com DeliveryMan)
EnterpriseDataManege	SIM	SIM (expandido para 3 tipos)
ProductDataManage	SIM	SIM
OrderDataManege	SIM	SIM
DeliveryDataManege	NÃO	Novo
Total	4	5

7. Exceções

Categoria	v1.0	v2.0
Exceções de Usuário	5	Mantidas
Exceções de Validação	8	Mantidas
Exceções de Empresa	5	Mantidas
Exceções de Produto	5	Mantidas
Exceções de Pedido	5	Mantidas
Exceções de Sistema	3	Mantidas
Exceções de Entrega	0	7 (NEW)
Total	41	48

8. Métodos na Facade

Categoria	v1.0	v2.0	Mudança
Usuário	7 métodos	7 métodos	Mantido

Categoria	v1.0	v2.0	Mudança
Empresa	6 métodos	6 métodos	Mantido
Produto	6 métodos	6 métodos	Mantido
Pedido	7 métodos	7 métodos	Mantido
Entrega	0	6	Novo
Total	26	32	

Novos Métodos de Entrega: 1. `criarEntrega(pedido, entregador, destino) → String` 2. `getEntrega(id, atributo) → String` 3. `getIdEntrega(pedido) → String` 4. `entregar(entrega) → void` 5. `obterPedido(entregador) → String` (com lógica de priorização) 6. `cadastrarEntregador(empresa, entregador) → void` (com ligação automática)

9. Fluxos de Operação

Fluxo v1.0	v2.0
Criar Usuário	SIM (+ DeliveryMan variant)
Criar Empresa	SIM (+ 3 tipos específicos)
Criar Produto	SIM
Criar Pedido	SIM
Liberação Pedido	SIM Expandido (distribui a entregadores)
Obter Pedido para Entrega	Novo
Criar Entrega	Novo
Finalizar Entrega	Novo

Fluxo v1.0	v2.0
ObterNÃO	Novo
Sta-	
tus	
de	
En-	
trega	

10. Camadas de Arquitetura

Antes (v1.0):

APRESENTAÇÃO (Main, EasyAccept, Facade)

NEGÓCIO (Core, Managers, Integrators)

VALIDAÇÃO (Validators)

DOMÍNIO (Models: User, Product, Order, Enterprise)

PERSISTÊNCIA (DataManagers → XML)

Depois (v2.0):

APRESENTAÇÃO (Main, EasyAccept, Facade)

ORQUESTRAÇÃO (Core + DeliveryManeger)

NEGÓCIO (5 Managers + 4 Integrators)

VALIDAÇÃO (Validators + DeliveryValidator)

DOMÍNIO (Models: User, Client, Owner, DeliveryMan)
 (Enterprise, Restaurant, Pharmacy, SuperMarket)
 (Product, Order, Delivery) ← NEW

PERSISTÊNCIA (5 DataManagers → XML)

Principais diferenças: - Camada de Domínio expandida (novo Delivery, múltiplas variantes de User/Enterprise) - Orquestração agora separada e

explícita (Core + DeliveryManager) - Nova camada de Validação (DeliveryValidator) - Persistência expandida (novo DeliveryDataManager)

Resumo Quantitativo

Métrica	v1.0	v2.0	Aumento
Classes de Usuário	3	4	+33%
Tipos de Empresa	1	3	+200%
Managers	4	5	+25%
DataManagers	4	5	+25%
Exceções	41	48	+17%
Métodos Facade	26	32	+23%
Estados de Pedido	3	5	+67%
Fluxos de Operação	7	13	+86%

Pontos Importantes de Implementação

1. Inicialização de Listas em DeliveryMan

```
public DeliveryMan() {  
    enterprises_Name_Address = new ArrayList<>();  
    deliveryByIdList = new ArrayList<>();  
    deliveryByIdPriorityList = new ArrayList<>();  
}
```

Por quê? Evita NullPointerException na desserialização XML.

2. Priorização de Entrega

```
public String getFirstOrder() throws NaoExistePedidoParaEntrega {  
    if (!deliveryByIdPriorityList.isEmpty()) {  
        return deliveryByIdPriorityList.getFirst(); // Farmácia sempre primeiro  
    } else if (!deliveryByIdList.isEmpty()) {  
        return deliveryByIdList.getFirst();  
    } else {  
        throw new NaoExistePedidoParaEntrega();  
    }  
}
```

Lógica: Farmácia → SEMPRE → Antes de qualquer outro pedido.

3. Remoção Correta de Pedidos

Quando um entregador pega um pedido:

```
// Remove de TODOS os entregadores
for (String devireyManEmail : deliverymanEmailList) {
    String deliveryManId = userDataManage.getIdByEmail(devireyManEmail);
    User user = userDataManage.getUserById(deliveryManId);
    ((DeliveryMan) user).removeOrder(orderId, priority);
}
```

4. Ligação Estado Delivery-Order

```
// Ao criar entrega
Order.setState("entregando");

// Ao finalizar entrega
Order.setState("entregue");
Entregador é liberado
```

Conclusão

As principais adições foram:

Adições de Arquitetura:

1. Nova entidade **Delivery** para rastreamento de entregas
2. Classe **DeliveryMan** para gerenciar entregadores
3. **3 tipos específicos de empresas** (Restaurant, Pharmacy, SuperMarket)
4. **DeliveryManager** para orquestração de entregas
5. **7 novas exceções** para validação de delivery
6. **6 novos métodos na Facade** para operações de delivery

Mantido Intacto:

- Arquitetura em camadas
- Padrões de design (Facade, Strategy, Template Method, etc)
- Métodos de usuário, empresa, produto, pedido
- Estrutura de persistência XML

Expandido:

- Hierarquia de usuários (+1 tipo)
- Hierarquia de empresas (+2 tipos)
- Fluxos de operação (+6 novos)
- Managers (+1)

- DataManagers (+1)
 - Estados de pedido (+2)
-